

Generative Models in Finance

Weeks 4–5: Flow Matching and Diffusion Models

Cristopher Salvi
Imperial College London

Spring 2026

Overview

- 1 Generative Modelling as Sampling
- 2 Flow Models
- 3 Diffusion Models
- 4 Constructing the Training Target
- 5 Training the Generative Model
- 6 A Guide to the Diffusion Model Literature
- 7 Building an Image Generator

Reference: P. Holderrieth & E. Erives, *An Introduction to Flow Matching and Diffusion Models*, MIT 6.S184, arXiv:2506.02070, 2025.

Part 1: Generative Modelling as Sampling

- Objects to generate (images, videos, molecules) are vectors $z \in \mathbb{R}^d$
- The diversity of valid objects is modelled by a **data distribution** p_{data} over $\mathbb{R}^d$
- **Generation = Sampling**: generating z means sampling $z \sim p_{\text{data}}$
- **Dataset**: i.i.d. samples $z_1, \dots, z_N \sim p_{\text{data}}$
- **Conditional generation**: sample $z \sim p_{\text{data}}(\cdot | y)$ for a conditioning variable y (e.g. text prompt). We focus on unconditional generation first
- **Goal**: Learn to transform samples from $p_{\text{init}} = \mathcal{N}(0, I_d)$ into samples from p_{data} , via simulation of a suitably constructed *differential equation*

Part 2: Flow Models — ODEs

- A **trajectory** is a function $X : [0, 1] \rightarrow \mathbb{R}^d$, $t \mapsto X_t$
- A **vector field** is a function

$$u : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d, \quad (x, t) \mapsto u_t(x),$$

specifying a velocity at every point x and time t

- An **ordinary differential equation (ODE)** with initial condition is:

$$\frac{d}{dt} X_t = u_t(X_t), \tag{4.1}$$

$$X_0 = x_0. \tag{4.2}$$

- Equation (4.1) says: the instantaneous rate of change of X_t is prescribed by the vector field u evaluated at the current position X_t

Flows

- The **flow** of the ODE is the function

$$\psi : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d, \quad (x_0, t) \mapsto \psi_t(x_0)$$

that maps each initial condition x_0 to the solution $X_t = \psi_t(x_0)$ at time t

- The flow satisfies:

$$\frac{d}{dt} \psi_t(x_0) = u_t(\psi_t(x_0)), \tag{4.3}$$

$$\psi_0(x_0) = x_0. \tag{4.4}$$

- **Intuition:** Vector fields, ODEs, and flows are three descriptions of the same object: *vector fields define ODEs whose solutions are flows*

Existence and Uniqueness

Theorem 4.1 (Flow existence and uniqueness)

*If $u : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$ is continuously differentiable with bounded derivative, then the ODE (4.1)–(4.2) has a unique solution given by a flow ψ_t . Moreover, ψ_t is a **diffeomorphism** for all t , i.e. ψ_t is continuously differentiable with continuously differentiable inverse ψ_t^{-1} .*

- Since we parameterise $u_t(x)$ with neural networks with smooth activations (e.g. GELU, SiLU), the assumptions are approximately satisfied in practice
- **Takeaway:** Flows exist, are unique, and are diffeomorphisms — they “warp” space in an invertible way

Example: Linear Vector Fields

Example 4.2 (Linear vector field)

Let $u_t(x) = -\theta x$ for $\theta > 0$. Then the flow is

$$\psi_t(x_0) = \exp(-\theta t) x_0. \quad (4.5)$$

- **Verification:** $\psi_0(x_0) = x_0$ ✓, and

$$\frac{d}{dt} \psi_t(x_0) = -\theta \exp(-\theta t) x_0 = -\theta \psi_t(x_0) = u_t(\psi_t(x_0)) \quad \checkmark$$

- This flow contracts all points exponentially towards the origin

Simulating an ODE: Euler Method

- In general, ψ_t cannot be computed in closed form
- The **Euler method**: initialise $X_0 = x_0$ and iterate

$$X_{t+h} = X_t + h u_t(X_t), \quad t = 0, h, 2h, \dots, 1 - h, \quad (4.6)$$

where $h = n^{-1}$ is the step size

- **Heun's method** (a second-order method):

$$\begin{aligned} X'_{t+h} &= X_t + h u_t(X_t) && \blacktriangleright \text{initial guess} \\ X_{t+h} &= X_t + \frac{h}{2} (u_t(X_t) + u_{t+h}(X'_{t+h})) && \blacktriangleright \text{corrected update} \end{aligned}$$

- Heun's method uses a predictor-corrector strategy: first guess, then average the velocities at the current and guessed states

Flow Models

- A **flow model** is defined by:

$$X_0 \sim p_{\text{init}}$$

► random initialisation

$$\frac{d}{dt}X_t = u_t^\theta(X_t)$$

► ODE

where $u_t^\theta : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$ is a neural network with parameters θ

- **Goal:** Choose θ so that the endpoint X_1 has distribution p_{data} , i.e.

$$X_1 \sim p_{\text{data}} \quad \Leftrightarrow \quad \psi_1^\theta(X_0) \sim p_{\text{data}}$$

- **Important:** The neural network parameterises the *vector field*, not the flow itself. The flow is obtained by simulating the ODE

Sampling from a Flow Model

Algorithm 1: Sampling from a Flow Model (Euler method)

Require: Neural network vector field u_t^θ , number of steps n

1. Set $t = 0$, step size $h = 1/n$
2. Draw $X_0 \sim p_{\text{init}}$
3. For $i = 1, \dots, n$ do:
4. $X_{t+h} = X_t + h u_t^\theta(X_t)$
5. $t \leftarrow t + h$
6. Return X_1

Part 3: Diffusion Models — Brownian Motion

Definition 4.3 (Brownian motion)

A *Brownian motion* (or *Wiener process*) $W = (W_t)_{0 \leq t \leq 1}$ is a stochastic process such that $W_0 = 0$, the trajectories $t \mapsto W_t$ are continuous, and:

- ① *Normal increments:* $W_t - W_s \sim \mathcal{N}(0, (t-s)I_d)$ for all $0 \leq s < t$
- ② *Independent increments:* For any $0 \leq t_0 < t_1 < \dots < t_n$, the increments $W_{t_1} - W_{t_0}, \dots, W_{t_n} - W_{t_{n-1}}$ are independent

- Approximate simulation: set $W_0 = 0$ and update

$$W_{t+h} = W_t + \sqrt{h} \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, I_d) \quad (4.7)$$

- Brownian paths are continuous but nowhere differentiable (a.s.)
- Brownian motion is the fundamental building block of stochastic calculus, analogous to the Gaussian distribution in probability theory

From ODEs to SDEs

- Rewrite the ODE via infinitesimal updates (using the definition of the derivative):

$$\frac{d}{dt}X_t = u_t(X_t) \quad \Leftrightarrow \quad X_{t+h} = X_t + h u_t(X_t) + h R_t(h)$$

where $\lim_{h \rightarrow 0} R_t(h) = 0$

- To make the dynamics *stochastic*, add a Brownian noise term:

$$X_{t+h} = X_t + \underbrace{h u_t(X_t)}_{\text{deterministic}} + \underbrace{\sigma_t(W_{t+h} - W_t)}_{\text{stochastic}} + h R_t(h) \quad (4.8)$$

where $\sigma_t \geq 0$ is the **diffusion coefficient**

- In symbolic (Itô) notation:

$$dX_t = u_t(X_t) dt + \sigma_t dW_t, \quad (4.9)$$

$$X_0 = x_0. \quad (4.10)$$

- **Caveat:** The “ dX_t ” notation is purely symbolic shorthand for (4.8). SDEs do not have flows in the ODE sense because trajectories are random

SDE Existence and Uniqueness

Theorem 4.4 (SDE solution existence and uniqueness)

If $u : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$ is continuously differentiable with bounded derivative and σ_t is continuous, then the SDE (4.9)–(4.10) has a unique stochastic process $(X_t)_{0 \leq t \leq 1}$ satisfying (4.8).

- A rigorous construction requires [stochastic integration](#) (Itô calculus); see e.g. Mao (2007)
- Every ODE is a special case of an SDE with $\sigma_t = 0$. Henceforth, [when we speak about SDEs, we consider ODEs as a special case](#)

The Ornstein–Uhlenbeck Process

Example 4.5 (Ornstein–Uhlenbeck process)

Let $u_t(x) = -\theta x$ for $\theta > 0$ and $\sigma_t = \sigma \geq 0$ constant. The SDE

$$dX_t = -\theta X_t dt + \sigma dW_t \quad (4.11)$$

defines an *Ornstein–Uhlenbeck (OU) process*.

- The drift $-\theta x$ pulls the process towards the origin (mean reversion)
- The noise σdW_t continuously perturbs it
- For $t \rightarrow \infty$, the process converges to $\mathcal{N}(0, \frac{\sigma^2}{2\theta} I_d)$
- For $\sigma = 0$: reduces to the deterministic flow $\psi_t(x_0) = e^{-\theta t} x_0$ from (4.5)
- The OU process is central to mathematical finance (Vasicek model) and to the theory of diffusion models

Simulating an SDE: Euler–Maruyama Method

- The **Euler–Maruyama method** is the SDE analogue of the Euler method for ODEs:

$$X_{t+h} = X_t + h u_t(X_t) + \sqrt{h} \sigma_t \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, I_d) \quad (4.12)$$

- At each step: take a small deterministic step in the direction $u_t(X_t)$, plus a random perturbation of size $\sqrt{h} \sigma_t$
- When $\sigma_t = 0$, this reduces to the Euler method (4.6)

Diffusion Models

- A **diffusion model** is defined by:

$$dX_t = u_t^\theta(X_t) dt + \sigma_t dW_t$$

$$X_0 \sim p_{\text{init}}$$

- ▶ SDE
- ▶ random initialisation

- It consists of:

- ▶ A neural network $u^\theta : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$, $(x, t) \mapsto u_t^\theta(x)$ with parameters θ
- ▶ A fixed diffusion coefficient $\sigma_t : [0, 1] \rightarrow [0, \infty)$

- **Goal:** $X_1 \sim p_{\text{data}}$
- A diffusion model with $\sigma_t = 0$ is a **flow model**

Sampling from a Diffusion Model

Algorithm 2: Sampling from a Diffusion Model (Euler–Maruyama)

Require: Neural network u_t^θ , number of steps n , diffusion coefficient σ_t

1. Set $t = 0$, step size $h = 1/n$
2. Draw $X_0 \sim p_{\text{init}}$
3. **For** $i = 1, \dots, n$ **do:**
4. Draw $\epsilon \sim \mathcal{N}(0, I_d)$
5. $X_{t+h} = X_t + h u_t^\theta(X_t) + \sigma_t \sqrt{h} \epsilon$
6. $t \leftarrow t + h$
7. **Return** X_1

Summary: Flow and Diffusion Models

Summary

A **diffusion model** consists of a neural network u^θ and a fixed diffusion coefficient σ_t :

Neural network: $u^\theta : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$, Fixed: $\sigma_t : [0, 1] \rightarrow [0, \infty)$.

To generate samples:

Initialisation: $X_0 \sim p_{\text{init}}$

Simulation: $dX_t = u_t^\theta(X_t) dt + \sigma_t dW_t$

Goal: $X_1 \sim p_{\text{data}}$

A diffusion model with $\sigma_t = 0$ is a **flow model**.

Part 4: Constructing the Training Target

- If we randomly initialise θ , simulating the ODE/SDE produces nonsense
- We need to *train* the neural network by minimising a loss:

$$\mathcal{L}(\theta) = \|u_t^\theta(x) - u_t^{\text{target}}(x)\|^2 \quad (4.13)$$

- The **training target** $u_t^{\text{target}} : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$ is a vector field whose corresponding ODE/SDE converts p_{init} into p_{data}
- **This part:** Derive a formula for u_t^{target}
- **Next part:** Design a tractable training algorithm to approximate u_t^{target}

Conditional Probability Paths

Definition 4.6 (Conditional probability path)

A *conditional (interpolating) probability path* is a family of distributions $p_t(\cdot \mid z)$ over $\mathbb{R}^d$, parameterised by $t \in [0, 1]$ and $z \in \mathbb{R}^d$, such that

$$p_0(\cdot \mid z) = p_{\text{init}}, \quad p_1(\cdot \mid z) = \delta_z, \quad \text{for all } z \in \mathbb{R}^d. \quad (4.14)$$

- Here δ_z is the Dirac delta at z (the distribution that always returns z)
- Intuitively, $p_t(\cdot \mid z)$ gradually converts a *single* data point z from noise p_{init} at $t = 0$ to a point mass at z at $t = 1$
- Think of this as a trajectory in the space of distributions, indexed by t

Marginal Probability Paths

- Every conditional path $p_t(\cdot | z)$ induces a **marginal probability path** p_t :
- **Sampling procedure:**

$$z \sim p_{\text{data}}, \quad x \sim p_t(\cdot | z) \quad \Rightarrow \quad x \sim p_t \quad (4.15)$$

- **Density:**

$$p_t(x) = \int p_t(x | z) p_{\text{data}}(z) dz \quad (4.16)$$

- By construction, the marginal path interpolates between p_{init} and p_{data} :

$$p_0 = p_{\text{init}} \quad \text{and} \quad p_1 = p_{\text{data}} \quad (4.17)$$

- **Note:** We can sample from p_t (via (4.15)), but we cannot evaluate the density $p_t(x)$ because the integral (4.16) is intractable

Gaussian Conditional Probability Paths

Example 4.7 (Gaussian conditional probability path)

Let α_t, β_t be *noise schedulers*: continuously differentiable, monotonic functions with $\alpha_0 = \beta_1 = 0$ and $\alpha_1 = \beta_0 = 1$. Define

$$p_t(\cdot \mid z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d). \quad (4.18)$$

- Check: $p_0(\cdot \mid z) = \mathcal{N}(0, \beta_0^2 I_d) = \mathcal{N}(0, I_d) = p_{\text{init}} \checkmark$
- Check: $p_1(\cdot \mid z) = \mathcal{N}(\alpha_1 z, 0) = \delta_z \checkmark$
- Sampling from the marginal path:

$$z \sim p_{\text{data}}, \quad \epsilon \sim \mathcal{N}(0, I_d) \quad \Rightarrow \quad x = \alpha_t z + \beta_t \epsilon \sim p_t \quad (4.19)$$

- Intuitively: more noise for smaller t (closer to $t = 0$, pure noise), less noise for larger t (closer to $t = 1$, pure data)
- **CondOT path**: $\alpha_t = t, \beta_t = 1 - t$ gives $p_t(\cdot \mid z) = \mathcal{N}(tz, (1 - t)^2 I_d)$

Conditional Vector Fields

- We now construct a vector field whose ODE trajectories follow the conditional probability path
- A **conditional vector field** $u_t^{\text{target}}(\cdot | z)$ is defined so that its ODE yields $X_t \sim p_t(\cdot | z)$:

$$X_0 \sim p_{\text{init}}, \quad \frac{d}{dt} X_t = u_t^{\text{target}}(X_t | z) \quad \Rightarrow \quad X_t \sim p_t(\cdot | z). \quad (4.20)$$

Conditional Gaussian Vector Field

Example 4.8 (Target ODE for Gaussian probability paths)

For $p_t(\cdot | z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$, with $\dot{\alpha}_t = \partial_t \alpha_t$ and $\dot{\beta}_t = \partial_t \beta_t$, the *conditional Gaussian vector field* is

$$u_t^{\text{target}}(x | z) = \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) z + \frac{\dot{\beta}_t}{\beta_t} x. \quad (4.21)$$

Proof. Define the conditional flow

$$\psi_t^{\text{target}}(x | z) = \alpha_t z + \beta_t x. \quad (4.22)$$

If $X_0 \sim \mathcal{N}(0, I_d) = p_{\text{init}}$, then

$$X_t = \psi_t^{\text{target}}(X_0 | z) = \alpha_t z + \beta_t X_0 \sim \mathcal{N}(\alpha_t z, \beta_t^2 I_d) = p_t(\cdot | z). \quad \checkmark$$

Conditional Gaussian Vector Field (Proof cont.)

It remains to extract $u_t^{\text{target}}(x | z)$ from ψ_t^{target} . By definition of a flow (4.3):

$$\frac{d}{dt} \psi_t^{\text{target}}(x | z) = u_t^{\text{target}}(\psi_t^{\text{target}}(x | z) | z) \quad \text{for all } x, z \in \mathbb{R}^d.$$

Differentiating (4.22):

$$\dot{\alpha}_t z + \dot{\beta}_t x = u_t^{\text{target}}(\alpha_t z + \beta_t x | z).$$

Substituting $x \rightarrow (x - \alpha_t z)/\beta_t$ (i.e. reparameterising):

$$\dot{\alpha}_t z + \dot{\beta}_t \cdot \frac{x - \alpha_t z}{\beta_t} = u_t^{\text{target}}(x | z).$$

Rearranging:

$$u_t^{\text{target}}(x | z) = \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) z + \frac{\dot{\beta}_t}{\beta_t} x. \quad \square$$

The Marginalization Trick

Theorem 4.9 (Marginalization trick)

Let $u_t^{\text{target}}(\cdot | z)$ be a conditional vector field satisfying (4.20). Then the *marginal vector field*

$$u_t^{\text{target}}(x) = \int u_t^{\text{target}}(x | z) \frac{p_t(x | z) p_{\text{data}}(z)}{p_t(x)} dz \quad (4.23)$$

yields ODE trajectories that follow the *marginal* probability path:

$$X_0 \sim p_{\text{init}}, \quad \frac{d}{dt} X_t = u_t^{\text{target}}(X_t) \quad \Rightarrow \quad X_t \sim p_t \quad (0 \leq t \leq 1). \quad (4.24)$$

In particular, $X_1 \sim p_{\text{data}}$.

- **Key insight:** We can construct the (intractable) marginal vector field from the (tractable) conditional vector field via (4.23)
- The proof uses the *continuity equation*

The Continuity Equation

Theorem 4.10 (Continuity equation)

Let u_t^{target} be a vector field with $X_0 \sim p_{\text{init}}$. Then $X_t \sim p_t$ for all $0 \leq t \leq 1$ if and only if

$$\partial_t p_t(x) = -\text{div}(p_t u_t^{\text{target}})(x) \quad \text{for all } x \in \mathbb{R}^d, \quad 0 \leq t \leq 1, \quad (4.25)$$

where $\text{div}(v_t)(x) = \sum_{i=1}^d \frac{\partial}{\partial x_i} v_t^i(x)$ is the *divergence* operator.

- **Interpretation:** The left side $\partial_t p_t(x)$ measures how probability density changes at x . The right side $-\text{div}(p_t u_t)$ measures the net inflow of probability mass via the vector field. Since mass is conserved, these must be equal
- A self-contained proof is given in the appendix slides. We use this result to prove the Marginalization Trick

Proof of the Marginalization Trick

We show that $u_t^{\text{target}}(x)$ defined in (4.23) satisfies the continuity equation (4.25).

$$\begin{aligned}\partial_t p_t(x) &\stackrel{(i)}{=} \partial_t \int p_t(x | z) p_{\text{data}}(z) \, dz \\ &= \int \partial_t p_t(x | z) p_{\text{data}}(z) \, dz \\ &\stackrel{(ii)}{=} \int -\text{div}(p_t(\cdot | z) u_t^{\text{target}}(\cdot | z))(x) p_{\text{data}}(z) \, dz \\ &\stackrel{(iii)}{=} -\text{div} \left(\int p_t(x | z) u_t^{\text{target}}(x | z) p_{\text{data}}(z) \, dz \right)\end{aligned}$$

where in (i) we used (4.16), in (ii) we used the continuity equation for the conditional path $p_t(\cdot | z)$, and in (iii) we swapped the integral and divergence.

Proof of the Marginalization Trick (cont.)

Continuing:

$$\begin{aligned} &\stackrel{(iv)}{=} -\operatorname{div} \left(p_t(x) \int u_t^{\text{target}}(x | z) \frac{p_t(x | z) p_{\text{data}}(z)}{p_t(x)} dz \right) \\ &\stackrel{(v)}{=} -\operatorname{div}(p_t u_t^{\text{target}})(x), \end{aligned}$$

where in (iv) we multiplied and divided by $p_t(x)$, and in (v) we used the definition (4.23).

The first and last expressions show that the continuity equation holds for u_t^{target} . By Theorem 4.10, this implies $X_t \sim p_t$ for $0 \leq t \leq 1$. □

The Score Function

Definition 4.11 (Score function)

The *marginal score function* of p_t is

$$\nabla \log p_t(x) = \frac{\nabla p_t(x)}{p_t(x)}.$$

The *conditional score function* of $p_t(\cdot \mid z)$ is $\nabla \log p_t(x \mid z)$.

- The marginal score can be expressed via conditional scores:

$$\nabla \log p_t(x) = \int \nabla \log p_t(x \mid z) \frac{p_t(x \mid z) p_{\text{data}}(z)}{p_t(x)} dz \quad (4.26)$$

- The conditional score is typically known analytically

Score Function for Gaussian Paths

Example 4.12 (Score for Gaussian conditional probability paths)

For $p_t(x | z) = \mathcal{N}(x; \alpha_t z, \beta_t^2 I_d)$, the conditional score is

$$\nabla \log p_t(x | z) = -\frac{x - \alpha_t z}{\beta_t^2}. \quad (4.27)$$

- **Proof.** The Gaussian density is $\mathcal{N}(x; \mu, \sigma^2 I_d) = (2\pi\sigma^2)^{-d/2} \exp\left(-\frac{\|x - \mu\|^2}{2\sigma^2}\right)$. Taking $\nabla_x \log$ gives $-\frac{x - \mu}{\sigma^2}$. With $\mu = \alpha_t z$ and $\sigma^2 = \beta_t^2$, the result follows
- **Key property:** The conditional score is *linear* in x . This is a special feature of Gaussian distributions

The Fokker–Planck Equation

Theorem 4.13 (Fokker–Planck equation)

Let p_t be a probability path and consider the SDE

$$X_0 \sim p_{\text{init}}, \quad dX_t = u_t(X_t) dt + \sigma_t dW_t.$$

Then X_t has distribution p_t for all $0 \leq t \leq 1$ if and only if the *Fokker–Planck equation* holds:

$$\partial_t p_t(x) = -\operatorname{div}(p_t u_t)(x) + \frac{\sigma_t^2}{2} \Delta p_t(x) \quad \text{for all } x \in \mathbb{R}^d, \quad 0 \leq t \leq 1, \quad (4.28)$$

where $\Delta w_t(x) = \sum_{i=1}^d \frac{\partial^2}{\partial x_i^2} w_t(x) = \operatorname{div}(\nabla w_t)(x)$ is the *Laplacian*.

- For $\sigma_t = 0$: reduces to the continuity equation (4.25)
- The additional term $\frac{\sigma_t^2}{2} \Delta p_t$ accounts for the spreading of probability mass due to Brownian noise (cf. the heat equation)

Proof of the Fokker–Planck Equation (Necessity)

Strategy: Use test functions $f : \mathbb{R}^d \rightarrow \mathbb{R}$ (smooth, compactly supported). We show the Fokker–Planck equation is necessary, i.e. if $X_t \sim p_t$ then (4.28) holds.

Step 1. From the SDE update (4.8), a second-order Taylor expansion of f around X_t gives:

$$\begin{aligned} f(X_{t+h}) - f(X_t) &\approx \nabla f(X_t)^T (h u_t(X_t) + \sigma_t(W_{t+h} - W_t)) \\ &\quad + \frac{1}{2} (h u_t(X_t) + \sigma_t(W_{t+h} - W_t))^T \nabla^2 f(X_t) \\ &\quad \times (h u_t(X_t) + \sigma_t(W_{t+h} - W_t)). \end{aligned}$$

Proof of the Fokker–Planck Equation (Necessity, cont.)

Step 2. Taking conditional expectation given X_t and using $\mathbb{E}[W_{t+h} - W_t \mid X_t] = 0$ and $(W_{t+h} - W_t) \mid X_t \sim \mathcal{N}(0, hI_d)$:

$$\begin{aligned}\mathbb{E}[f(X_{t+h}) - f(X_t) \mid X_t] &= h \nabla f(X_t)^T u_t(X_t) \\ &\quad + \frac{h}{2} \sigma_t^2 \text{trace}(\nabla^2 f(X_t)) + O(h^2) \\ &= h \nabla f(X_t)^T u_t(X_t) + \frac{h}{2} \sigma_t^2 \Delta f(X_t) + O(h^2).\end{aligned}$$

Here we used $\mathbb{E}_{\epsilon \sim \mathcal{N}(0, I_d)}[\epsilon^T A \epsilon] = \text{trace}(A)$.

Proof of the Fokker–Planck Equation (Necessity, cont.)

Step 3. Divide by h , take $h \rightarrow 0$, and take full expectation:

$$\partial_t \mathbb{E}[f(X_t)] = \mathbb{E} \left[\nabla f(X_t)^T u_t(X_t) + \frac{\sigma_t^2}{2} \Delta f(X_t) \right].$$

Using $X_t \sim p_t$, write as integrals:

$$\partial_t \int f(x) p_t(x) dx = \int \nabla f(x)^T u_t(x) p_t(x) dx + \int \frac{\sigma_t^2}{2} \Delta f(x) p_t(x) dx.$$

Proof of the Fokker–Planck Equation (Necessity, cont.)

Step 4. Apply integration by parts:

- $\int \nabla f^T(p_t u_t) dx = - \int f \operatorname{div}(p_t u_t) dx$
- $\int \Delta f \cdot p_t dx = \int f \cdot \Delta p_t dx$

Substituting:

$$\int f(x) \partial_t p_t(x) dx = \int f(x) \left(-\operatorname{div}(p_t u_t)(x) + \frac{\sigma_t^2}{2} \Delta p_t(x) \right) dx.$$

Since this holds for all test functions f , we conclude

$$\partial_t p_t(x) = -\operatorname{div}(p_t u_t)(x) + \frac{\sigma_t^2}{2} \Delta p_t(x). \quad \square$$

Sufficiency follows from uniqueness of solutions to parabolic PDEs: both p_t and the true distribution q_t of X_t satisfy (4.28) with the same initial condition $p_0 = q_0 = p_{\text{init}}$, so $p_t = q_t$ for all t .

The SDE Extension Trick

Theorem 4.14 (SDE extension trick)

Define the conditional and marginal vector fields $u_t^{\text{target}}(x | z)$ and $u_t^{\text{target}}(x)$ as before. Then for any diffusion coefficient $\sigma_t \geq 0$, the SDE

$$X_0 \sim p_{\text{init}}, \quad dX_t = \left[u_t^{\text{target}}(X_t) + \frac{\sigma_t^2}{2} \nabla \log p_t(X_t) \right] dt + \sigma_t dW_t \quad (4.29)$$

satisfies $X_t \sim p_t$ for $0 \leq t \leq 1$. In particular, $X_1 \sim p_{\text{data}}$.

- For $\sigma_t = 0$: recovers the ODE $dX_t = u_t^{\text{target}}(X_t) dt$
- The same identity holds replacing marginal quantities by conditional ones
- **Key insight:** Adding noise ($\sigma_t > 0$) requires a correction term $\frac{\sigma_t^2}{2} \nabla \log p_t$ to maintain the same marginal path

Proof of the SDE Extension Trick

We verify the Fokker–Planck equation (4.28) for the SDE (4.29):

$$\begin{aligned}\partial_t p_t(x) &\stackrel{(i)}{=} -\operatorname{div}(p_t u_t^{\text{target}})(x) \\ &\stackrel{(ii)}{=} -\operatorname{div}(p_t u_t^{\text{target}})(x) - \frac{\sigma_t^2}{2} \Delta p_t(x) + \frac{\sigma_t^2}{2} \Delta p_t(x) \\ &\stackrel{(iii)}{=} -\operatorname{div}(p_t u_t^{\text{target}})(x) - \operatorname{div}\left(\frac{\sigma_t^2}{2} \nabla p_t\right)(x) + \frac{\sigma_t^2}{2} \Delta p_t(x) \\ &\stackrel{(iv)}{=} -\operatorname{div}(p_t u_t^{\text{target}})(x) - \operatorname{div}\left(p_t \cdot \frac{\sigma_t^2}{2} \nabla \log p_t\right)(x) + \frac{\sigma_t^2}{2} \Delta p_t(x) \\ &\stackrel{(v)}{=} -\operatorname{div}\left(p_t \left[u_t^{\text{target}} + \frac{\sigma_t^2}{2} \nabla \log p_t\right]\right)(x) + \frac{\sigma_t^2}{2} \Delta p_t(x),\end{aligned}$$

where (i): continuity equation; (ii): add and subtract; (iii): $\Delta p_t = \operatorname{div}(\nabla p_t)$; (iv): $\nabla p_t = p_t \nabla \log p_t$; (v): linearity of div . This is the Fokker–Planck equation for the SDE (4.29). □

Langevin Dynamics

Remark 4.15 (Langevin dynamics)

A famous special case: if the probability path is static ($p_t = p$ for all t), set $u_t^{\text{target}} = 0$ to obtain:

$$dX_t = \frac{\sigma_t^2}{2} \nabla \log p(X_t) dt + \sigma_t dW_t. \quad (4.30)$$

- Since $p_t = p$ is static, $\partial_t p_t = 0$, and p is a *stationary distribution* of the SDE
- If $X_0 \sim p$, then $X_t \sim p$ for all $t \geq 0$
- Even if $X_0 \not\sim p$, under mild conditions $X_t \rightarrow p$ as $t \rightarrow \infty$ (convergence to equilibrium)
- Langevin dynamics is the foundation of many MCMC methods in Bayesian statistics, molecular dynamics, and sampling algorithms

Summary: Constructing the Training Target

Summary

- 1 Choose a **conditional probability path** $p_t(x | z)$ with $p_0(\cdot | z) = p_{\text{init}}$, $p_1(\cdot | z) = \delta_z$
- 2 Find a **conditional vector field** $u_t^{\text{target}}(x | z)$ whose ODE follows $p_t(\cdot | z)$
- 3 The **marginal vector field** $u_t^{\text{target}}(x) = \int u_t^{\text{target}}(x | z) \frac{p_t(x|z)p_{\text{data}}(z)}{p_t(x)} dz$ converts p_{init} into p_{data}
- 4 Extending to SDEs: $dX_t = [u_t^{\text{target}}(X_t) + \frac{\sigma_t^2}{2} \nabla \log p_t(X_t)] dt + \sigma_t dW_t$

For **Gaussian probability paths** $p_t(\cdot | z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$:

$$u_t^{\text{target}}(x | z) = \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) z + \frac{\dot{\beta}_t}{\beta_t} x, \quad \nabla \log p_t(x | z) = -\frac{x - \alpha_t z}{\beta_t^2}.$$

Part 5: Training — Flow Matching

- Consider a flow model: $X_0 \sim p_{\text{init}}, dX_t = u_t^\theta(X_t) dt$
- We want $u_t^\theta \approx u_t^{\text{target}}$. A natural objective is the **flow matching loss**:

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t \sim \text{Unif}, x \sim p_t} \left[\|u_t^\theta(x) - u_t^{\text{target}}(x)\|^2 \right] \quad (4.31)$$

- **Problem:** We don't know $u_t^{\text{target}}(x)$ — it involves an intractable integral (4.23)
- **Idea:** Replace the marginal target with the *conditional* target $u_t^{\text{target}}(x | z)$, which we know analytically

The Conditional Flow Matching Loss

- Define the **conditional flow matching (CFM)** loss:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t \sim \text{Unif}, z \sim p_{\text{data}}, x \sim p_t(\cdot | z)} \left[\|u_t^\theta(x) - u_t^{\text{target}}(x | z)\|^2 \right] \quad (4.32)$$

- Note: we regress against the *conditional* vector field $u_t^{\text{target}}(x | z)$ instead of the marginal $u_t^{\text{target}}(x)$
- This loss is *tractable*: sample $t \sim \text{Unif}[0, 1]$, $z \sim p_{\text{data}}$, $x \sim p_t(\cdot | z)$ (add noise), and compute $\|u_t^\theta(x) - u_t^{\text{target}}(x | z)\|^2$
- But does minimising $\mathcal{L}_{\text{CFM}}$ also minimise $\mathcal{L}_{\text{FM}}$?

Equivalence of Flow Matching Losses

Theorem 4.16

The marginal and conditional flow matching losses are equal up to a constant independent of θ :

$$\mathcal{L}_{\text{FM}}(\theta) = \mathcal{L}_{\text{CFM}}(\theta) + C,$$

where C does not depend on θ . Therefore $\nabla_{\theta} \mathcal{L}_{\text{FM}}(\theta) = \nabla_{\theta} \mathcal{L}_{\text{CFM}}(\theta)$.

- **Consequence:** Minimising the tractable $\mathcal{L}_{\text{CFM}}$ with SGD is equivalent to minimising the intractable $\mathcal{L}_{\text{FM}}$
- For the optimal θ^* : $u_t^{\theta^*} = u_t^{\text{target}}$ (with infinite capacity)

Proof of Loss Equivalence

Expand $\mathcal{L}_{\text{FM}}$ by completing the square:

$$\begin{aligned}\mathcal{L}_{\text{FM}}(\theta) &\stackrel{(i)}{=} \mathbb{E}_{t,x \sim p_t} \left[\|u_t^\theta(x)\|^2 - 2u_t^\theta(x)^T u_t^{\text{target}}(x) + \|u_t^{\text{target}}(x)\|^2 \right] \\ &\stackrel{(ii)}{=} \mathbb{E}_{t,x \sim p_t} \left[\|u_t^\theta(x)\|^2 \right] - 2\mathbb{E}_{t,x \sim p_t} \left[u_t^\theta(x)^T u_t^{\text{target}}(x) \right] + C_1\end{aligned}$$

where (i) is $\|a - b\|^2 = \|a\|^2 - 2a^T b + \|b\|^2$, (ii) defines $C_1 = \mathbb{E}[\|u_t^{\text{target}}(x)\|^2]$ (independent of θ).

Using the sampling procedure (4.15) for p_t :

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t,z \sim p_{\text{data}}, x \sim p_t(\cdot|z)} \left[\|u_t^\theta(x)\|^2 \right] - 2\mathbb{E}_{t,x \sim p_t} \left[u_t^\theta(x)^T u_t^{\text{target}}(x) \right] + C_1$$

Proof of Loss Equivalence (cont.)

The key step: rewrite the cross term using (4.23):

$$\begin{aligned}\mathbb{E}_{t,x \sim p_t} \left[u_t^\theta(x)^T u_t^{\text{target}}(x) \right] &\stackrel{(i)}{=} \int_0^1 \int p_t(x) u_t^\theta(x)^T \left[\int u_t^{\text{target}}(x | z) \frac{p_t(x | z) p_{\text{data}}(z)}{p_t(x)} dz \right] dx dt \\ &\stackrel{(ii)}{=} \int_0^1 \int \int u_t^\theta(x)^T u_t^{\text{target}}(x | z) p_t(x | z) p_{\text{data}}(z) dz dx dt \\ &\stackrel{(iii)}{=} \mathbb{E}_{t,z \sim p_{\text{data}}, x \sim p_t(\cdot | z)} \left[u_t^\theta(x)^T u_t^{\text{target}}(x | z) \right]\end{aligned}$$

where (i) plugs in (4.23), (ii) cancels $p_t(x)$, (iii) recognises the expectation. This is the crucial step: the marginal vector field is replaced by the conditional one.

Proof of Loss Equivalence (cont.)

Substituting back:

$$\begin{aligned}\mathcal{L}_{\text{FM}}(\theta) &= \mathbb{E}_{t,z,x \sim p_t(\cdot|z)} \left[\|u_t^\theta(x)\|^2 - 2u_t^\theta(x)^T u_t^{\text{target}}(x|z) \right] + C_1 \\&= \mathbb{E}_{t,z,x \sim p_t(\cdot|z)} \left[\|u_t^\theta(x)\|^2 - 2u_t^\theta(x)^T u_t^{\text{target}}(x|z) \right. \\&\quad \left. + \|u_t^{\text{target}}(x|z)\|^2 - \|u_t^{\text{target}}(x|z)\|^2 \right] + C_1 \\&= \underbrace{\mathbb{E}_{t,z,x} \left[\|u_t^\theta(x) - u_t^{\text{target}}(x|z)\|^2 \right]}_{=\mathcal{L}_{\text{CFM}}(\theta)} + \underbrace{C_1 - \mathbb{E} \left[\|u_t^{\text{target}}(x|z)\|^2 \right]}_{=:C}\end{aligned}$$

Since C is independent of θ , we have $\mathcal{L}_{\text{FM}}(\theta) = \mathcal{L}_{\text{CFM}}(\theta) + C$. □

Flow Matching for Gaussian CondOT Paths

- For $\alpha_t = t$, $\beta_t = 1 - t$ (CondOT path):

$$\dot{\alpha}_t = 1, \quad \dot{\beta}_t = -1, \quad u_t^{\text{target}}(x \mid z) = z - \epsilon \quad \text{where } x = tz + (1 - t)\epsilon$$

- The CFM loss becomes:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t \sim \text{Unif}, z \sim p_{\text{data}}, \epsilon \sim \mathcal{N}(0, I_d)} \left[\|u_t^\theta(tz + (1 - t)\epsilon) - (z - \epsilon)\|^2 \right] \quad (4.33)$$

- **Remarkable simplicity:** Sample data z , noise ϵ , time t ; interpolate $x = tz + (1 - t)\epsilon$; regress $u_t^\theta(x)$ against $z - \epsilon$
- This is used by *Stable Diffusion 3*, *Meta Movie Gen Video*, and many other state-of-the-art models

Flow Matching Training Algorithm

Algorithm 3: Flow Matching Training (Gaussian CondOT)

Require: Dataset $z \sim p_{\text{data}}$, neural network u_t^θ

1. **For** each mini-batch **do**:
2. Sample z from dataset
3. Sample $t \sim \text{Unif}[0, 1]$
4. Sample $\epsilon \sim \mathcal{N}(0, I_d)$
5. Set $x = tz + (1 - t)\epsilon$
6. Compute loss $\mathcal{L}(\theta) = \|u_t^\theta(x) - (z - \epsilon)\|^2$
7. Update θ via gradient descent on $\mathcal{L}(\theta)$

For general Gaussian paths $p_t(\cdot | z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$: set $x \sim p_t(\cdot | z)$ and regress against $u_t^{\text{target}}(x | z)$ from (4.21).

Score Matching

- To extend from ODEs to SDEs, we also need to approximate the **marginal score** $\nabla \log p_t$
- We train a **score network** $s_t^\theta : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$ via the **score matching loss**:

$$\mathcal{L}_{SM}(\theta) = \mathbb{E}_{t \sim \text{Unif}, x \sim p_t} \left[\|s_t^\theta(x) - \nabla \log p_t(x)\|^2 \right] \quad (4.34)$$

- Again intractable (we don't know $\nabla \log p_t(x)$). Define the **conditional score matching loss**:

$$\mathcal{L}_{CSM}(\theta) = \mathbb{E}_{t \sim \text{Unif}, z \sim p_{\text{data}}, x \sim p_t(\cdot | z)} \left[\|s_t^\theta(x) - \nabla \log p_t(x | z)\|^2 \right] \quad (4.35)$$

Equivalence of Score Matching Losses

Theorem 4.17

The score matching loss equals the conditional score matching loss up to a constant:

$$\mathcal{L}_{\text{SM}}(\theta) = \mathcal{L}_{\text{CSM}}(\theta) + C,$$

where C is independent of θ .

- **Proof:** The formula for $\nabla \log p_t$ in (4.26) has exactly the same structure as the formula for u_t^{target} in (4.23). The proof is identical to Theorem 4.16, replacing u_t^{target} with $\nabla \log p_t$
- At the optimum: $s_t^{\theta^*} = \nabla \log p_t$

Denoising Score Matching for Gaussian Paths

- For Gaussian paths $p_t(\cdot | z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$, the conditional score (4.27) gives:

$$\mathcal{L}_{\text{CSM}}(\theta) = \mathbb{E}_{t,z,\epsilon \sim \mathcal{N}(0, I_d)} \left[\left\| s_t^\theta(\alpha_t z + \beta_t \epsilon) + \frac{\epsilon}{\beta_t} \right\|^2 \right] \quad (4.36)$$

- The network learns to predict the noise ϵ that was used to corrupt z — this is **denoising score matching**
- Alternatively, reparameterise via a **noise predictor** $\epsilon_t^\theta(x) := -\beta_t s_t^\theta(x)$:

$$\mathcal{L}_{\text{DDPM}}(\theta) = \mathbb{E}_{t,z,\epsilon} \left[\left\| \epsilon_t^\theta(\alpha_t z + \beta_t \epsilon) - \epsilon \right\|^2 \right] \quad (4.37)$$

- This is the loss used in **Denoising Diffusion Probabilistic Models** (DDPM, Ho et al. 2020)

Score Matching Training Algorithm

Algorithm 4: Score Matching Training (Gaussian paths)

Require: Dataset $z \sim p_{\text{data}}$, score network s_t^θ or noise predictor ϵ_t^θ

1. **For** each mini-batch **do**:
2. Sample z from dataset
3. Sample $t \sim \text{Unif}[0, 1]$
4. Sample $\epsilon \sim \mathcal{N}(0, I_d)$
5. Set $x_t = \alpha_t z + \beta_t \epsilon$
6. Compute loss $\mathcal{L}(\theta) = \|s_t^\theta(x_t) + \epsilon/\beta_t\|^2$
 (Alternatively: $\mathcal{L}(\theta) = \|\epsilon_t^\theta(x_t) - \epsilon\|^2$)
7. Update θ via gradient descent on $\mathcal{L}(\theta)$

Conversion Between Vector Field and Score

Proposition 4.18 (Conversion formula for Gaussian probability paths)

For $p_t(\cdot | z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$, the conditional (resp. marginal) vector field and score are related by:

$$u_t^{\text{target}}(x | z) = \left(\beta_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \dot{\beta}_t \beta_t \right) \nabla \log p_t(x | z) + \frac{\dot{\alpha}_t}{\alpha_t} x, \quad (4.38)$$

$$u_t^{\text{target}}(x) = \left(\beta_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \dot{\beta}_t \beta_t \right) \nabla \log p_t(x) + \frac{\dot{\alpha}_t}{\alpha_t} x. \quad (4.39)$$

The vector field $u_t^{\text{target}}(x)$ in (4.39) is called the *probability flow ODE*.

Proof of the Conversion Formula

From (4.21) and (4.27):

$$\begin{aligned} u_t^{\text{target}}(x | z) &= \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) z + \frac{\dot{\beta}_t}{\beta_t} x \\ &\stackrel{(i)}{=} \left(\beta_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \dot{\beta}_t \beta_t \right) \left(\frac{\alpha_t z - x}{\beta_t^2} \right) + \frac{\dot{\alpha}_t}{\alpha_t} x \\ &= \left(\beta_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \dot{\beta}_t \beta_t \right) \nabla \log p_t(x | z) + \frac{\dot{\alpha}_t}{\alpha_t} x, \end{aligned}$$

where (i) is algebra (rearranging z in terms of $\frac{\alpha_t z - x}{\beta_t^2}$ and x). The marginal identity (4.39) follows by integrating both sides against $\frac{p_t(x|z)p_{\text{data}}(z)}{p_t(x)}$ and using (4.23) and (4.26). □

Consequence: For Gaussian paths, learning either u_t^θ or s_t^θ suffices — they are interconvertible. No need to train both separately.

Sampling with Both Networks

- Given a trained score s_t^θ and vector field u_t^θ (or one derived from the other), sample via the SDE for any $\sigma_t \geq 0$:

$$X_0 \sim p_{\text{init}}, \quad dX_t = \left[u_t^\theta(X_t) + \frac{\sigma_t^2}{2} s_t^\theta(X_t) \right] dt + \sigma_t dW_t \quad (4.40)$$

- $\sigma_t = 0$: **deterministic** sampling (flow/probability flow ODE)
- $\sigma_t > 0$: **stochastic** sampling (diffusion)
- In theory, every σ_t gives $X_1 \sim p_{\text{data}}$ at perfect training
- In practice, the optimal σ_t is found empirically (balances numerical and training errors)
- Using the conversion formula with a single network:

$$dX_t = \left[\left(\beta_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \dot{\beta}_t \beta_t + \frac{\sigma_t^2}{2} \right) s_t^\theta(X_t) + \frac{\dot{\alpha}_t}{\alpha_t} X_t \right] dt + \sigma_t dW_t \quad (4.41)$$

Summary: Training

Summary

Flow Matching: Train u_t^θ via the conditional flow matching loss:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{z \sim p_{\text{data}}, t \sim \text{Unif}, x \sim p_t(\cdot | z)} \left[\|u_t^\theta(x) - u_t^{\text{target}}(x | z)\|^2 \right].$$

Score Matching: Train s_t^θ via conditional score matching:

$$\mathcal{L}_{\text{CSM}}(\theta) = \mathbb{E}_{z \sim p_{\text{data}}, t \sim \text{Unif}, x \sim p_t(\cdot | z)} \left[\|s_t^\theta(x) - \nabla \log p_t(x | z)\|^2 \right].$$

For Gaussian paths, these are equivalent (up to reparameterisation). After training, sample via ODE or SDE.

Part 6: Literature Guide — Discrete vs. Continuous Time

- The first denoising diffusion models (Sohl-Dickstein et al. 2015; Ho et al. 2020; Song & Ermon 2019) used **discrete-time** Markov chains with time steps $t = 0, 1, 2, 3, \dots$
- The loss function was approximated via an **evidence lower bound (ELBO)** — only a lower bound on the true log-likelihood
- Song et al. (2021) showed these constructions are essentially approximations of *continuous-time* SDEs
- In continuous time, the ELBO becomes *tight*: Theorem 4.16 and Theorem 4.17 are equalities (not lower bounds)
- **Advantage of continuous time**: Mathematically cleaner; one can control the simulation error via ODE/SDE samplers post-training; no need to commit to a time discretisation before training

Forward Processes and Probability Paths

- Early diffusion model papers did not use the term “probability path”. Instead, they constructed a **forward process** — an SDE that gradually *noises* the data:

$$\bar{X}_0 = z, \quad d\bar{X}_t = u_t^{\text{forw}}(\bar{X}_t) dt + \sigma_t^{\text{forw}} d\bar{W}_t \quad (4.42)$$

- Starting from data $z \sim p_{\text{data}}$, the forward process converges to $\mathcal{N}(0, I_d)$ for $t \rightarrow \infty$
- The conditional distribution $\bar{X}_t \mid \bar{X}_0 = z$ is a conditional probability path $\bar{p}_t(\cdot \mid z)$
- For affine drift $u_t^{\text{forw}}(x) = a_t x$, the conditional distribution is Gaussian:

$$\bar{X}_t \mid \bar{X}_0 = z \sim \mathcal{N}(\alpha_t z, \beta_t^2 I), \quad \alpha_t = \exp\left(\int_0^t a_r dr\right), \quad \beta_t^2 = \alpha_t^2 \int_0^t \frac{(\sigma_r^{\text{forw}})^2}{\alpha_r^2} dr.$$

- **Key point:** A forward process is a *specific way* of constructing a Gaussian probability path. The term “probability path” (introduced by flow matching) is more general and simpler
- **Important:** Forward processes use an **inverted time convention**: $\bar{p}_0(\cdot \mid z) = \delta_z$ (data at $t = 0$, noise at $t = \infty$), opposite to our convention

Time-Reversal vs. Fokker–Planck

- The original diffusion model literature derived the training target via the **time-reversal** of the forward process (Anderson, 1982)
- A **time-reversal** $(X_t)_{0 \leq t \leq T}$ of $(\bar{X}_t)$ is an SDE with the same distribution over trajectories but with time running backwards:

$$\mathbb{P}[\bar{X}_{t_1} \in A_1, \dots, \bar{X}_{t_n} \in A_n] = \mathbb{P}[X_{T-t_1} \in A_1, \dots, X_{T-t_n} \in A_n]$$

- Anderson's formula:

$$dX_t = \left[-u_t(X_t) + \sigma_t^2 \nabla \log p_t(X_t) \right] dt + \sigma_t dW_t$$

where $u_t(x) = u_{T-t}^{\text{forw}}(x)$ and $\sigma_t = \sigma_{T-t}^{\text{forw}}$

- This is a specific instance of Theorem 4.14 (after accounting for time inversion)
- **Modern approach** (as in this course): Derive the training target directly via the Fokker–Planck/continuity equation, avoiding the need for time-reversal entirely. This was pioneered by flow matching (Lipman et al. 2022; Liu et al. 2022) and stochastic interpolants (Albergo et al. 2023)

Flow Matching and Stochastic Interpolants

- **Flow matching** (Lipman et al. 2022; Liu et al. 2022): Restricts to ODEs (i.e. $\sigma_t = 0$). Key contribution: showing that one does *not* need a forward process or time-reversal to train generative models scalably
- **Stochastic interpolants** (Albergo et al. 2023): Extends flow matching to include the SDE case via “Langevin dynamics” extension (Theorem 4.14)
- Both frameworks use probability paths instead of forward processes, and construct training targets via the Fokker–Planck equation
- **Advantages over classical diffusion:**
 - ▶ Simpler training framework (direct MSE regression)
 - ▶ Greater generality: arbitrary p_{init} (not just Gaussian) and arbitrary probability paths (not just Gaussian)
 - ▶ Classical denoising diffusion models are recovered as a special case (Gaussian initial distribution + Gaussian probability path)

Denoising Diffusion Models: Recap

- **Denoising diffusion models** are diffusion models with Gaussian probability paths
 $p_t(\cdot | z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$
- For this special case:
 - ▶ It suffices to learn *either* the vector field u_t^θ or the score s_t^θ (Proposition 4.18)
 - ▶ Training can use flow matching *or* (denoising) score matching — both lead to the same model up to reparameterisation
- After training, one can sample via:
 - ▶ ODE (deterministic, probability flow ODE)
 - ▶ SDE (stochastic, for any $\sigma_t \geq 0$)
- **Limitation vs. flow matching:** Denoising diffusion requires $p_{\text{init}} = \mathcal{N}(0, I_d)$ and Gaussian probability paths. Flow matching allows arbitrary p_{init} and arbitrary paths

Literature Roadmap

Alternative Formulations in the Literature

- 1 **Discrete-time:** Approximations of SDEs via discrete-time Markov chains (DDPM, NCSN)
- 2 **Inverted time convention:** $t = 0 = \text{data}$, $t = T = \text{noise}$ (vs. our $t = 0 = \text{noise}$, $t = 1 = \text{data}$)
- 3 **Forward process:** A specific way of constructing (Gaussian) probability paths via a noising SDE
- 4 **Time-reversal:** Training target constructed by reversing the forward SDE, rather than via the Fokker–Planck equation

All formulations lead to the same models and training losses. The probability path / Fokker–Planck approach (used in this course) is the most general and arguably the simplest.

Part 7: Building an Image Generator — Guidance

- So far: unconditional generation ($X_1 \sim p_{\text{data}}$)
- Now: generate objects **conditioned on** a variable $y \in \mathcal{Y}$ (e.g. text prompt, class label)
- Goal: sample from $p_{\text{data}}(\cdot | y)$ for any y
- We use the term **guided** (rather than “conditional”) to avoid confusion with “conditional probability path” $p_t(\cdot | z)$
- A **guided diffusion model** has a vector field $u_t^\theta(\cdot | y) : \mathbb{R}^d \times \mathcal{Y} \times [0, 1] \rightarrow \mathbb{R}^d$ and samples via:

$$X_0 \sim p_{\text{init}}$$

$$dX_t = u_t^\theta(X_t | y) dt + \sigma_t dW_t$$

$$\text{Goal: } X_1 \sim p_{\text{data}}(\cdot | y)$$

Guided Flow Matching

- For fixed y , use $p_{\text{data}}(\cdot | y)$ as the data distribution and apply CFM:

$$\mathcal{L}_{\text{CFM}}^{\text{guided}}(\theta) = \mathbb{E}_{(z,y) \sim p_{\text{data}}, t \sim \text{Unif}, x \sim p_t(\cdot | z)} \left[\|u_t^\theta(x | y) - u_t^{\text{target}}(x | z)\|^2 \right] \quad (4.43)$$

- Note: the conditional probability path $p_t(\cdot | z)$ and target $u_t^{\text{target}}(x | z)$ do *not* depend on y — only the network input does
- In practice: the dataloader returns pairs (z, y)
- **Empirical observation:** This procedure is theoretically correct but often produces images that don't fit the prompt y well enough

Classifier-Free Guidance: Derivation

- For Gaussian paths, using Proposition 4.18, the guided vector field can be rewritten as:

$$u_t^{\text{target}}(x | y) = a_t x + b_t \nabla \log p_t(x | y) \quad (4.44)$$

for explicit $(a_t, b_t) = \left(\frac{\dot{\alpha}_t}{\alpha_t}, \frac{\dot{\alpha}_t \beta_t^2 - \dot{\beta}_t \beta_t \alpha_t}{\alpha_t} \right)$

- By Bayes' rule:

$$\nabla \log p_t(x | y) = \nabla \log p_t(x) + \nabla \log p_t(y | x) \quad (4.45)$$

since $\nabla \log p_t(y) = 0$ w.r.t. the gradient in x

- Idea:** Scale up the $\nabla \log p_t(y | x)$ term by a **guidance scale** $w > 1$:

$$\tilde{u}_t(x | y) = u_t^{\text{target}}(x) + w b_t \nabla \log p_t(y | x)$$

Classifier-Free Guidance: The Formula

Applying (4.45) again:

$$\begin{aligned}\tilde{u}_t(x | y) &= u_t^{\text{target}}(x) + w b_t (\nabla \log p_t(x | y) - \nabla \log p_t(x)) \\ &= u_t^{\text{target}}(x) - (w a_t x + w b_t \nabla \log p_t(x)) + (w a_t x + w b_t \nabla \log p_t(x | y)) \\ &= (1 - w) u_t^{\text{target}}(x) + w u_t^{\text{target}}(x | y).\end{aligned}$$

Denoting the unconditional model as $u_t^{\text{target}}(x | \emptyset) := u_t^{\text{target}}(x)$:

Classifier-Free Guidance (CFG)

$$\tilde{u}_t(x | y) = (1 - w) u_t^{\text{target}}(x | \emptyset) + w u_t^{\text{target}}(x | y). \quad (4.46)$$

- $w = 1$: standard guided generation
- $w > 1$: amplifies the conditioning signal (heuristic, but empirically very effective)
- For $w \neq 1$: $\tilde{u}_t \neq u_t^{\text{target}}(x | y)$, so the distribution is no longer exactly $p_{\text{data}}(\cdot | y)$

CFG Training

- **Key insight:** We can train both $u_t^\theta(x | y)$ and $u_t^\theta(x | \emptyset)$ with a *single* network
- Augment the label set with a special “no label” token $\emptyset$
- During training, replace y with $\emptyset$ with probability η (a hyperparameter)
- The **CFG conditional flow matching objective**:

$$\mathcal{L}_{\text{CFM}}^{\text{CFG}}(\theta) = \mathbb{E}_{\square} \left[\|u_t^\theta(x | y) - u_t^{\text{target}}(x | z)\|^2 \right] \quad (4.47)$$

where $\square = (z, y) \sim p_{\text{data}}$, $t \sim \text{Unif}$, $x \sim p_t(\cdot | z)$, replace $y = \emptyset$ with prob. η

- At inference: compute $\tilde{u}_t^\theta(x | y) = (1 - w)u_t^\theta(x | \emptyset) + w u_t^\theta(x | y)$ and simulate ODE

CFG Training Algorithm

Algorithm 5: CFG Training (Gaussian CondOT path)

Require: Paired dataset $(z, y) \sim p_{\text{data}}$, neural network u_t^θ

1. **For** each mini-batch **do**:
2. Sample (z, y) from dataset
3. Sample $t \sim \text{Unif}[0, 1]$
4. Sample $\epsilon \sim \mathcal{N}(0, I_d)$
5. Set $x = \alpha_t z + \beta_t \epsilon$
6. With probability η : set $y \leftarrow \emptyset$
7. Compute loss $\mathcal{L}(\theta) = \|u_t^\theta(x \mid y) - (\dot{\alpha}_t z + \dot{\beta}_t \epsilon)\|^2$
8. Update θ via gradient descent

CFG for Diffusion Models

- The same idea applies to score-based diffusion models
- By Bayes' rule: $\nabla \log p_t(x | y) = \nabla \log p_t(x) + \nabla \log p_t(y | x)$
- For guidance scale $w > 1$, define the **classifier-free guided score**:

$$\tilde{s}_t(x | y) = (1 - w) \nabla \log p_t(x | \emptyset) + w \nabla \log p_t(x | y) \quad (4.48)$$

CFG Training for Diffusion Models

- The **guided conditional score matching** objective:

$$\mathcal{L}_{\text{CSM}}^{\text{guided}}(\theta) = \mathbb{E}_{\square} \left[\|s_t^\theta(x | y) - \nabla \log p_t(x | z)\|^2 \right] \quad (4.49)$$

where $\square = (z, y) \sim p_{\text{data}}$, $t \sim \text{Unif}$, $x \sim p_t(\cdot | z)$

- The **CFG conditional score matching** objective (with label dropout):

$$\mathcal{L}_{\text{CSM}}^{\text{CFG}}(\theta) = \mathbb{E}_{\square} \left[\|s_t^\theta(x | y) - \nabla \log p_t(x | z)\|^2 \right] \quad (4.50)$$

where $\square = (z, y) \sim p_{\text{data}}$, $t \sim \text{Unif}$, $x \sim p_t(\cdot | z)$, replace $y = \emptyset$ with prob. η

- At inference, combine both networks and sample via:

$$\begin{aligned} \tilde{s}_t^\theta(x | y) &= (1 - w) s_t^\theta(x | \emptyset) + w s_t^\theta(x | y) \\ \tilde{u}_t^\theta(x | y) &= (1 - w) u_t^\theta(x | \emptyset) + w u_t^\theta(x | y) \\ dX_t &= \left[\tilde{u}_t^\theta(X_t | y) + \frac{\sigma_t^2}{2} \tilde{s}_t^\theta(X_t | y) \right] dt + \sigma_t dW_t \end{aligned}$$

Neural Network Architectures

- The neural network $u_t^\theta(x | y)$ must take 3 inputs: $x \in \mathbb{R}^d$, $y \in \mathcal{Y}$, $t \in [0, 1]$, and output a vector in $\mathbb{R}^d$
- For low-dimensional problems: a **multi-layer perceptron (MLP)** suffices (concatenate x , y , t and pass through)
- For images/videos: specialised architectures are needed
- Two dominant architectures:
 - 1 **U-Net** (Ronneberger et al. 2015): Convolutional; input and output have image shape
 - 2 **Diffusion Transformer (DiT)** (Peebles & Xie 2023): Attention-based; patchify the image and process patches with a transformer

U-Net Architecture

- An image $x_t \in \mathbb{R}^{C \times H \times W}$ is processed through:
 - ① **Encoders** $\mathcal{E}_i$: Convolutional blocks that progressively *downsample* (reduce spatial resolution, increase channels)
 - ② **Midcoder** $\mathcal{M}$: Processes the latent representation at lowest resolution
 - ③ **Decoders** $\mathcal{D}_i$: Convolutional blocks that *upsample* back to original resolution
- **Residual connections** (skip connections) link each encoder to its corresponding decoder — this preserves fine-grained spatial information
- Time t and conditioning y are injected into each block (typically via learned embeddings added to intermediate activations)
- The “U” shape: channels increase / spatial dims decrease going down, then reverse going up

Diffusion Transformers (DiT)

- Based on [Vision Transformers \(ViT\)](#): divide the image into patches, embed each patch, and process with self-attention
- Advantages over U-Nets: purely attention-based (no convolutions); scales more naturally with compute
- Time t and class label y are injected via [adaptive layer norm \(adaLN\)](#) — modulating the scale and shift parameters of layer normalisation
- For text conditioning: use [cross-attention](#) between image patch tokens and text embedding tokens
- DiTs are used in modern large-scale models (Stable Diffusion 3, Movie Gen Video)

Working in Latent Space

- For high-resolution images (1000×1000 pixels $\rightarrow \sim 3$ million dimensions), training a diffusion model in pixel space is prohibitively expensive
- **Latent diffusion models:** Train a (variational) **autoencoder** to compress images into a low-dimensional latent space, then train the diffusion model *in the latent space*
- Pipeline:
 - 1 **Encode:** $z_{\text{latent}} = \text{Encoder}(z_{\text{image}})$ — compress training images
 - 2 **Train:** Learn u_t^θ in the latent space
 - 3 **Sample:** Generate X_1 in latent space, then $z_{\text{image}} = \text{Decoder}(X_1)$
- A good autoencoder filters out semantically meaningless detail and retains perceptually relevant features
- Nearly all state-of-the-art image and video models use latent diffusion

Encoding the Conditioning Variable

- **Class labels** ($y \in \{0, \dots, N\}$): Learn an embedding vector for each class; these embeddings are parameters of u_t^θ
- **Text prompts**: Use a pre-trained text encoder to map y_{raw} (text) to a continuous embedding $y \in \mathbb{R}^{d_y}$
- **CLIP** (Contrastive Language–Image Pre-training, Radford et al. 2021): Trains a shared image–text embedding space. Provides a coarse, global embedding
- For richer conditioning: use a pre-trained language model (e.g. T5) to produce a *sequence* of embeddings, and condition via **cross-attention**
- Modern models combine multiple text encoders to capture both coarse semantics and fine-grained details

Case Study: Stable Diffusion 3

- **Architecture:** Multi-modal Diffusion Transformer (MM-DiT) — a modified DiT that attends to both image patches and text embeddings
- **Training:** Conditional flow matching (CondOT path), with classifier-free guidance (label dropping)
- **Text conditioning:** Three types of text embeddings (CLIP-G/14, CLIP-L/14, T5-XXL) — combining coarse and fine-grained representations
- **Latent space:** Trained in the latent space of a pre-trained autoencoder
- **Scale:** Largest model has 8 billion parameters
- **Sampling:** 50 Euler steps with CFG weight $w \in [2, 5]$

Case Study: Meta Movie Gen Video

- **Data:** Videos $x \in \mathbb{R}^{T \times C \times H \times W}$ (additional temporal dimension)
- **Training:** Conditional flow matching (CondOT path) in the latent space of a **temporal autoencoder** (TAE) with compression factor 8 in each dimension
- **Architecture:** DiT-like backbone, patchified along time and space; self-attention over image patches + cross-attention with text embeddings
- **Text conditioning:** Three types (UL2 for reasoning, ByT5 for character-level detail, MetaCLIP for image-text alignment)
- **Scale:** Largest model has **30 billion parameters**
- **Takeaway:** The mathematical framework (flow matching + Gaussian CondOT + CFG) is *the same* as for images; the main engineering challenges are handling the temporal dimension and scaling

Summary: Building an Image Generator

Recipe for a Large-Scale Image/Video Generator

- ➊ **Probability path:** Gaussian CondOT $p_t(\cdot | z) = \mathcal{N}(tz, (1 - t)^2 I_d)$
- ➋ **Architecture:** Diffusion Transformer (DiT / MM-DiT)
- ➌ **Training:** Conditional flow matching with CFG (label dropout probability η)
- ➍ **Latent space:** Train in the latent space of a pre-trained autoencoder
- ➎ **Text conditioning:** Pre-trained text encoders (CLIP, T5) injected via cross-attention
- ➏ **Sampling:** Euler ODE with CFG weight $w > 1$

End of Weeks 4–5